بحث تعليمي · لغة C#

مُحدِّدات الوصول المركّبة و Reflection

دراسة واضحة لثلاثة مفاهيم تتحكم في «مَن يرى ماذا» داخل الكود، وكيف يمكن فحص الكود نفسه أثناء التشغيل.

① `protected internal`

② `private protected`

③ `Reflection`

نوفمبر 2026

1 مقدمة ومحتويات البحث

في C# كل عضو داخل الكلاس (حقل، دالة، خاصية) له «مستوى وصول» يحدد من يستطيع استخدامه. أغلب المبرمجين يحفظون `public` و `private` فقط، ثم يصطدمون بمحدِّدين يبدوان متشابهين تمامًا: `protected internal` و `private protected`. الفرق بينهما صغير في الكتابة، لكنه جوهري في السلوك.

أما **Reflection** فهو موضوع مختلف في الظاهر، لكنه مرتبط بالموضوعين الأولين: فهو الأداة التي تقرأ هذه المحدِّدات أثناء التشغيل، وتستطيع في بعض الحالات تجاوزها.

1 مراجعة سريعة لمحدِّدات الوصول

2 `protected internal` — الاتحاد («أو»)

3 `private protected` — التقاطع («و»)

4 جدول المقارنة الحاسم بين الاثنين

5 `Reflection` — التعريف والاستخدامات والأمثلة

6 العلاقة بين Reflection ومحدِّدات الوصول

7 الأخطاء الشائعة والخلاصة

2 مراجعة سريعة: محدِّدات الوصول

قبل المركّبة، نحتاج الأساس. كلمة **Assembly** تعني الوحدة المترجَمة (ملف `.dll` أو `.exe`)، وغالبًا هي مشروع واحد في Visual Studio. هذا المفهوم هو مفتاح فهم كل ما سيأتي.

المحدِّد	من يستطيع الوصول؟
`public`	أي كود، في أي مكان
`private`	نفس الكلاس فقط (الافتراضي لأعضاء الكلاس)
`protected`	نفس الكلاس + الكلاسات الوارثة منه (حتى في Assembly آخر)
`internal`	أي كود داخل نفس الـ Assembly فقط
`protected internal`	نفس الـ Assembly **أو** الوارث (من أي Assembly)
`private protected`	الوارث **و** في نفس الـ Assembly معًا

3 `protected internal`

التعريف

العضو المعلَّم بـ `protected internal` يمكن الوصول إليه من **أحد شرطين** (يكفي تحقق واحد):

- الكود موجود في **نفس الـ Assembly** — سواء كان وارثًا أم لا.
- الكود موجود في كلاس **وارث** — حتى لو كان في Assembly مختلف.

> **القاعدة الذهبية**
>
> `protected internal` = `protected` **أو** `internal`. هو أوسع من كل منهما منفردًا.

متى نستخدمه؟

عند بناء مكتبة (`class library`) تريد أن يستخدم باقي كلاساتها العضو بحرية، وفي نفس الوقت تسمح لمن يرث منها خارج المكتبة بالتخصيص (`override`) أو الاستخدام. هو في الواقع محدِّد «مفتوح نسبيًا» ويُستخدم بحذر.

4 `private protected`

التعريف

أُضيف في **C# 7.2**. العضو المعلَّم به لا يمكن الوصول إليه إلا إذا تحقق **الشرطان معًا**:

- الكود في كلاس **وارث** (أو نفس الكلاس).
- والكود موجود في **نفس الـ Assembly**.

> **القاعدة الذهبية**
>
> `private protected` = `protected` **و** `internal`. هو أضيق من كل منهما منفردًا.

متى نستخدمه؟

عندما تريد أن يرث كلاسات مشروعك فقط من هذا الكلاس ويستخدموا العضو، بينما تمنع أي مبرمج خارجي يرث منه عبر مكتبتك من رؤيته. مناسب لتفاصيل داخلية في هرم الوراثة لا يجب أن تُكشف للخارج.

5 المقارنة الحاسمة

الجدول التالي يجيب عن السؤال الأشهر في المقابلات: من يرى العضو في كل حالة؟

موقع الكود الذي يحاول الوصول	`protected internal`	`private protected`
داخل نفس الكلاس	✔	✔
كلاس وارث — نفس الـ Assembly	✔	✔
كلاس غير وارث — نفس الـ Assembly	✔	✘
كلاس وارث — Assembly مختلف	✔	✘
كلاس غير وارث — Assembly مختلف	✘	✘

مثال عملي

```
// ===== Assembly A (المكتبة) =====
public class Base
{
    protected internal string Info = "data";   // protected OR internal
    private protected int    Secret = 42;       // protected AND internal
}

class SameAssemblyOther      // غير وارث، نفس الـ Assembly
{
    void Test()
    {
        var b = new Base();
        Console.WriteLine(b.Info);     // ✔ مسموح (internal)
        // Console.WriteLine(b.Secret); // ✘ خطأ: ليس وارثًا
    }
}

// ===== Assembly B (مشروع آخر يرجع للمكتبة) =====
class Child : Base
{
    void Test()
    {
        Console.WriteLine(Info);       // ✔ مسموح (protected)
        // Console.WriteLine(Secret);  // ✘ خطأ: Assembly مختلف
    }
}
```

الخطأ الأكثر شيوعًا

كثيرون يقرؤون `protected internal` على أنه «وارث **و** داخل الـ Assembly». هذا خطأ؛ هذا هو `private protected`. للتذكر: **الأول يبدأ بـ protected لكنه «أو»**، والثاني يبدأ بـ private (أي مقيَّد) وهو «و».

6 Reflection

التعريف

Reflection (الانعكاس) هو قدرة البرنامج على فحص بنيته هو أثناء التشغيل: معرفة الأنواع (Types) والدوال والخصائص والحقول المتاحة، بل واستدعاؤها وإنشاء كائنات منها دون معرفتها وقت كتابة الكود.

يقع في مساحة الأسماء `System.Reflection`.

المكوّنات الأساسية

Type

نقطة البداية. تُحصل عليها بـ `typeof(X)` أو `obj.GetType()`، وتصف الكلاس كاملًا.

Assembly

يمثل ملف الـ dll/exe، ويسمح بتحميله (`Assembly.Load`) واستخراج أنواعه.

MethodInfo / PropertyInfo / FieldInfo

كائنات تصف الأعضاء، ويمكن عبرها القراءة والكتابة والاستدعاء.

Activator / BindingFlags

`Activator.CreateInstance` لإنشاء كائن ديناميكيًا، و`BindingFlags` لتحديد نوع الأعضاء المطلوبة.

مثال شامل

```
using System.Reflection;

class Person
{
    public string Name { get; set; } = "";
    private int _age = 30;
    public void Hello() => Console.WriteLine($"Hi {Name}");
}

var type = typeof(Person);

// 1) استكشاف الخصائص
foreach (var p in type.GetProperties())
    Console.WriteLine($"{p.Name} : {p.PropertyType.Name}");

// 2) إنشاء كائن وتعيين خاصية واستدعاء دالة
object obj = Activator.CreateInstance(type)!;
type.GetProperty("Name")!.SetValue(obj, "Anwr");
type.GetMethod("Hello")!.Invoke(obj, null);   // Hi Anwr

// 3) قراءة حقل private (!)
var f = type.GetField("_age", BindingFlags.NonPublic | BindingFlags.Instance);
Console.WriteLine(f!.GetValue(obj));              // 30
```

أين يُستخدم فعليًا؟

- **الـ Attributes:** قراءة الوسوم مثل `[Obsolete]` و`[Required]` وتفعيل سلوك بناءً عليها.
- **أنظمة الـ Plugins:** تحميل ملفات dll من مجلد واستكشاف ما بها دون الرجوع إليها وقت الترجمة.
- **أطر العمل:** الـ Serialization (مثل JSON) والـ ORMs (مثل Entity Framework) وحقن الاعتماديات (`Dependency Injection`) وأطر الاختبار؛ كلها تعتمد عليه داخليًا.
- **أدوات المطوّرين:** مستكشفات الكائنات والـ debuggers.

الثمن: لا شيء مجاني

العيب	التفسير
أبطأ	الاستدعاء عبر Reflection أبطأ بوضوح من الاستدعاء المباشر؛ يُتفادى داخل الحلقات الساخنة أو يُخزَّن الـ `MethodInfo` مؤقتًا (caching).
لا أمان وقت الترجمة	الأسماء نصوص (`"Name"`)؛ خطأ إملائي يظهر وقت التشغيل لا وقت الترجمة. تحسّنه `nameof()`.
يكسر التغليف	يستطيع الوصول إلى الأعضاء الخاصة، فيجعل الاعتماد على تفاصيل داخلية هشًّا.
مشاكل التحجيم و AOT	أدوات تقليص الكود (`trimming`) و`Native AOT` قد تحذف ما يُستخدم عبر Reflection فقط.

7 العلاقة بين Reflection ومحدِّدات الوصول

محدِّدات الوصول تُفرض **وقت الترجمة** (`compile-time`) من المترجم. أما Reflection فيعمل **وقت التشغيل**، وبالتالي بمجرد إعطائه `BindingFlags.NonPublic` يستطيع رؤية الأعضاء `private` و`internal` و`protected internal` و`private protected` جميعها وقراءتها وتعديلها، ما دامت صلاحيات التشغيل تسمح (وهي تسمح في التطبيقات العادية).

> **النتيجة العملية**
>
> محدِّدات الوصول **ليست آلية أمان**، بل أداة تصميم تحمي الكود من الاستخدام الخاطئ بالخطأ. من يحتاج حماية حقيقية لأسرار (كلمات مرور، مفاتيح) لا يعتمد على `private`.

8 الأخطاء الشائعة والخلاصة

أخطاء تتكرر

- الخلط بين `protected internal` («أو») و`private protected` («و»).
- اختيار `protected internal` افتراضيًا «للاحتياط»؛ هو في الحقيقة يوسّع الكشف. الأفضل البدء بالأضيق ثم التوسيع عند الحاجة.
- استخدام Reflection لحل مشكلة يكفي لها `interface` أو `generics` أو `polymorphism`.
- استدعاء Reflection داخل حلقة متكررة بدون تخزين مؤقت للنتائج.

الخلاصة في ثلاثة أسطر

`protected internal` — أوسع: نفس الـ Assembly **أو** الوارثون.

`private protected` — أضيق: الوارثون **و** نفس الـ Assembly فقط.

`Reflection` — قوي ومرن لكنه بطيء ويتجاوز التغليف؛ يُستخدم عند الحاجة الفعلية، لا كحل افتراضي.

مراجع للاستزادة

Microsoft Learn — Access Modifiers (C# Reference)

Microsoft Learn — Reflection in .NET

Microsoft Learn — What's new in C# 7.2 (private protected)